AN0214
Application Note

AT32 BLDC Sensorless Quick Start Guide

Introduction

This application note describes the quick-start debugging SOP for sensorless BLDC motor operation using Artery's motor development board. Artery offers multiple motor development boards, and this document uses the AT-MOTOR-EVB as an example for illustration.

AT-MOTOR-EVB Applicable products:

Part number	AT32F402/405 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32L021 series
	AT32F45x series
	AT32M412/M416 series

Contents

List of tables

List of figures

1 Motor control library algorithm

This application note primarily covers the following content: Target motor type: Three-phase permanent magnet synchronous motor (brushless DC motor, BLDC)

Control methods:

- 120° square wave commutation

Three-phase PWM method:

- 120° conduction PWM control

Phase current sensing method:

- 1-shunt current sensing and reconstruction method

Initial rotor position estimation mode:

- Three-phase voltage vector mode

Rotor position estimation in 120° square wave control mode:

- BEMF zero crossing point detection by ADC
- BEMF zero crossing point detection by comparator

Sensorless 120° square-wave commutation mode:

- 120° square wave voltage control
- Torque control (120° square wave current control)
- Speed control

Auto tuning:

- Automatic identification of motor winding parameters
- Current PI controller parameter auto tuning

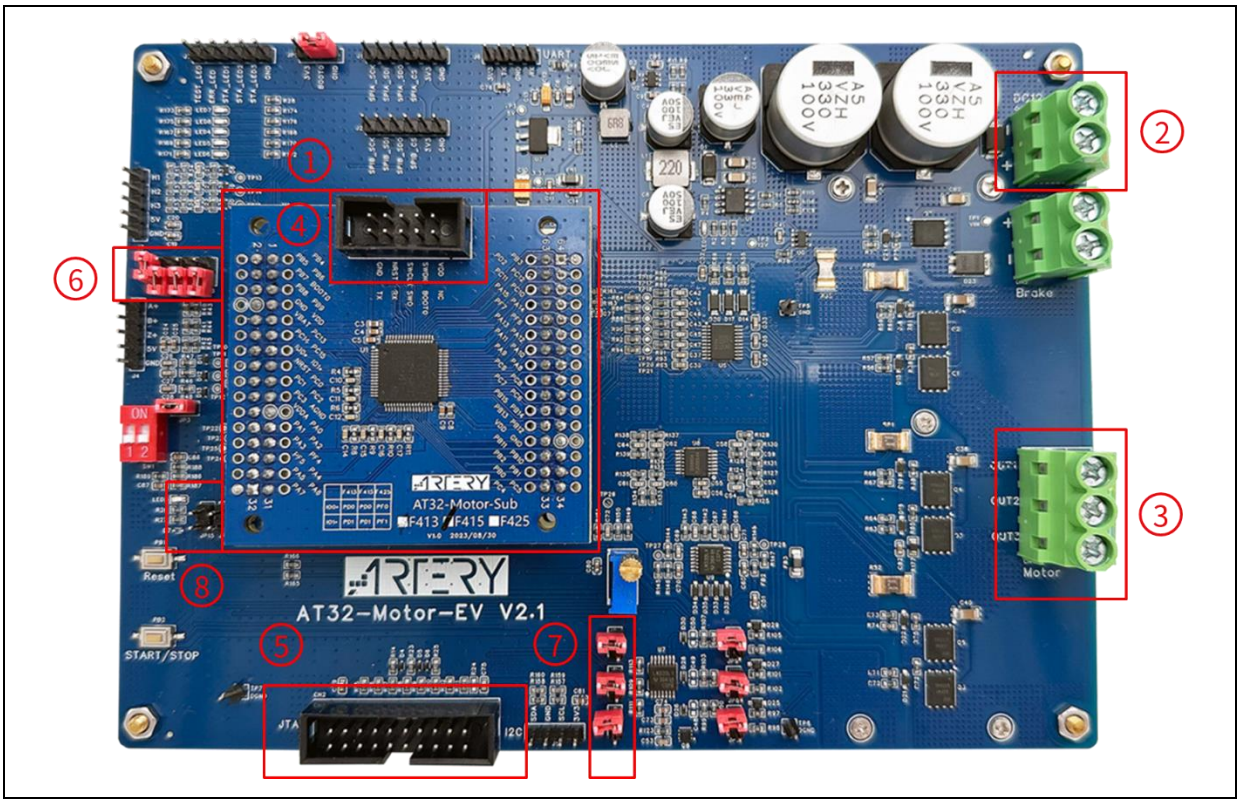
2 Environment requirements

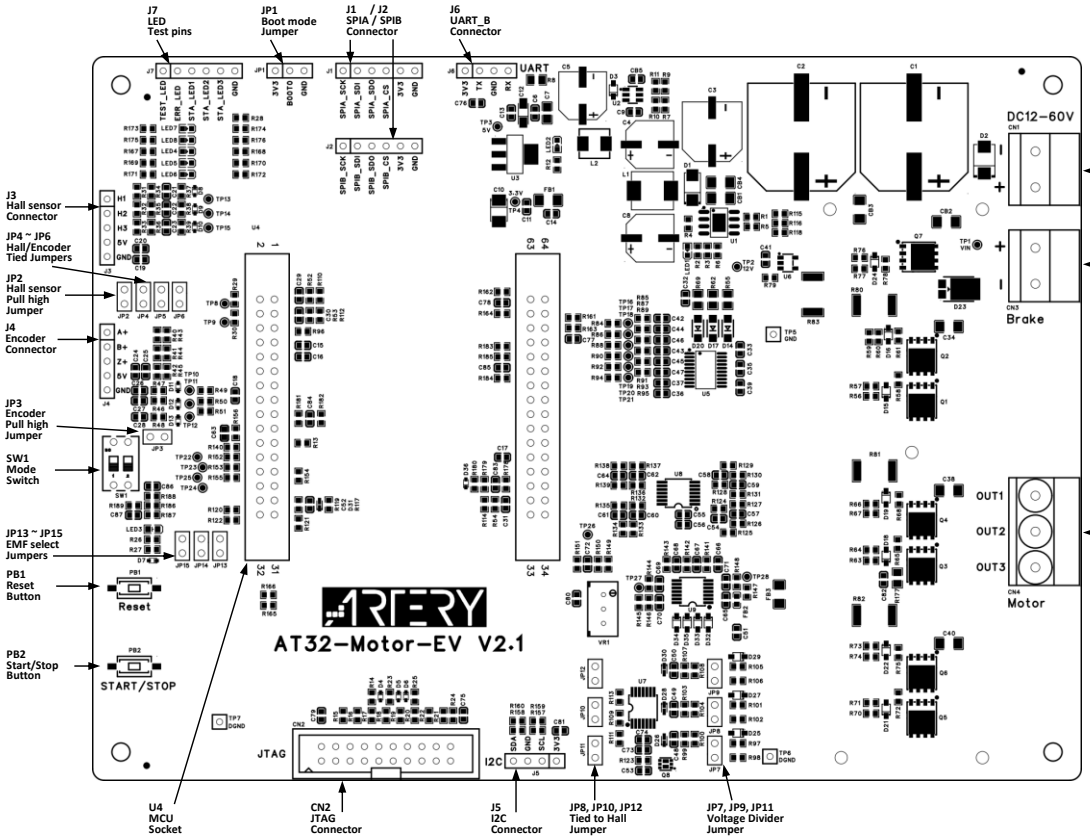
2.1 Hardware requirements

A PMSM (BLDC) motor, AT-Link or third-party debugger and a motor control board (AT-MOTOR-EVB) are required. Refer to UM0011 AT32_LV_Motor_Control_EVB_User_Manual for relevant hardware configurations.

- PMSM (BLDC) motor
- Debugger
- Motor control board: AT-MOTOR-EVB (embeds an AT32F413RCT7 MCU)

Figure 1. AT-MOTOR-EVB evaluation board





① AT32 MCU transfer board (part number: AT32F413RCT7)

② Power input connector CN1: connected to 24 V

③ Driver output connector CN4: connected to three-phase motor U, V and W (yellow, green and blue) leads, respectively

④ AT-Link connector: choose this connector or J-Link connector

⑤ J-Link connector: choose this connector or AT-Link connector

⑥ Jumper: JP2, JP3, JP4, JP5, JP6 OPEN

⑦ Jumper: JP8, JP10, JP12 OPEN (BEMF zero crossing point detection by ADC)
JP8, JP10, JP12 CLOSED (BEMF zero crossing point detection by CMP)

⑧ Jumper settings: JP13, JP14, JP15 CLOSED

2.2 Software requirements

1) Open the “bldc_1shunt_sensorless” demo project;

2) Run ArteryMotorMonitor.exe directly (installation not required);

3) In Keil, go to ***Options - Read/Only Memory Areas*** to set ROM according to the Flash memory size of each AT32 MCU (see Table 1). For example, for AT32F413RCT7 with a 256KB Flash memory, its IROM1 start address is 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is 0x803F800 with a size of 0x800 (see Figure 2). In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 3). In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 4).

Table 1. Flash memory size and ROM configuration

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: Keil complier version 5 is recommended since AT32 BSP source code does not support V6.15 compiler.

Figure 2. AT32F413RCT7 ROM settings (Keil)

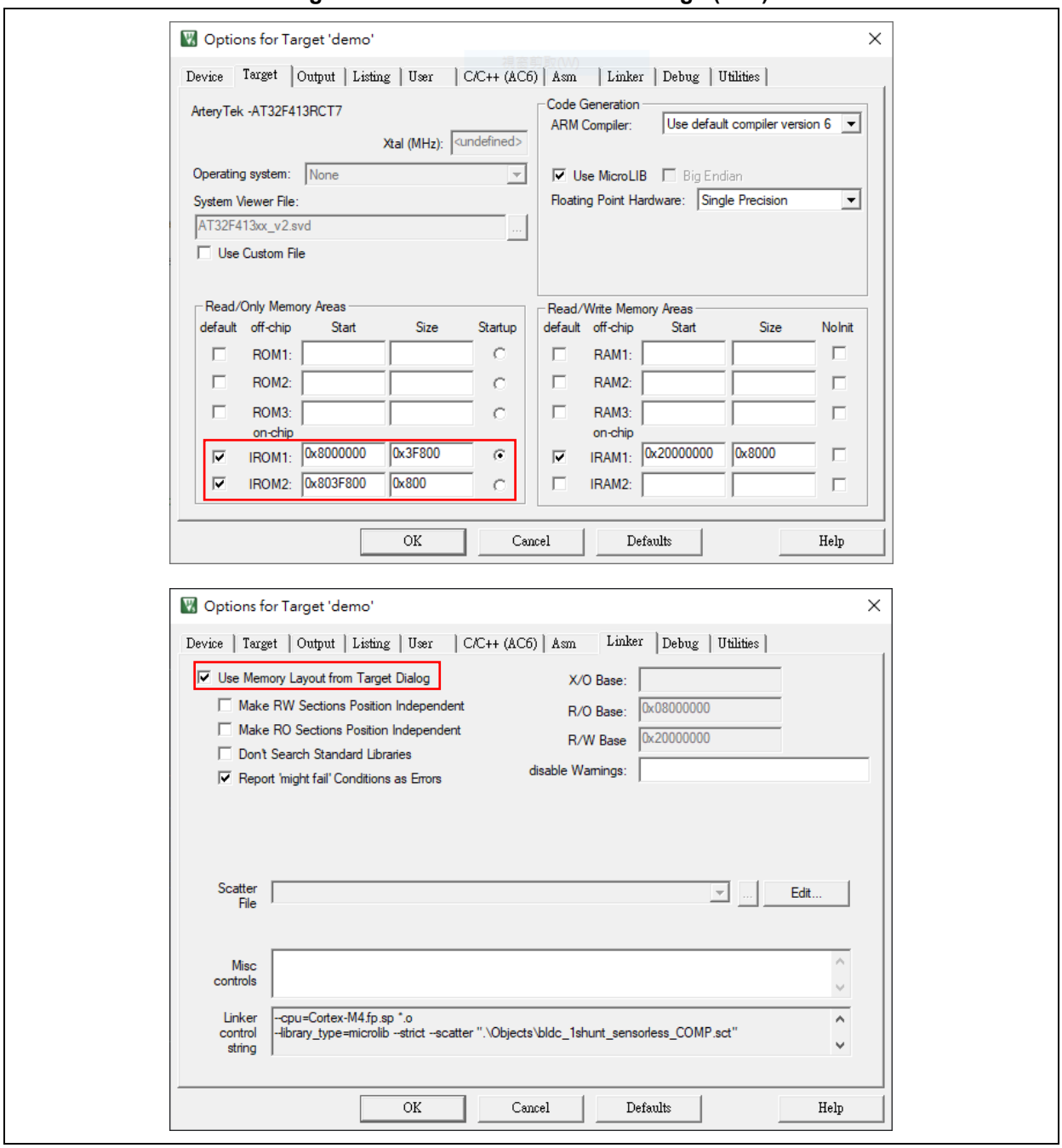
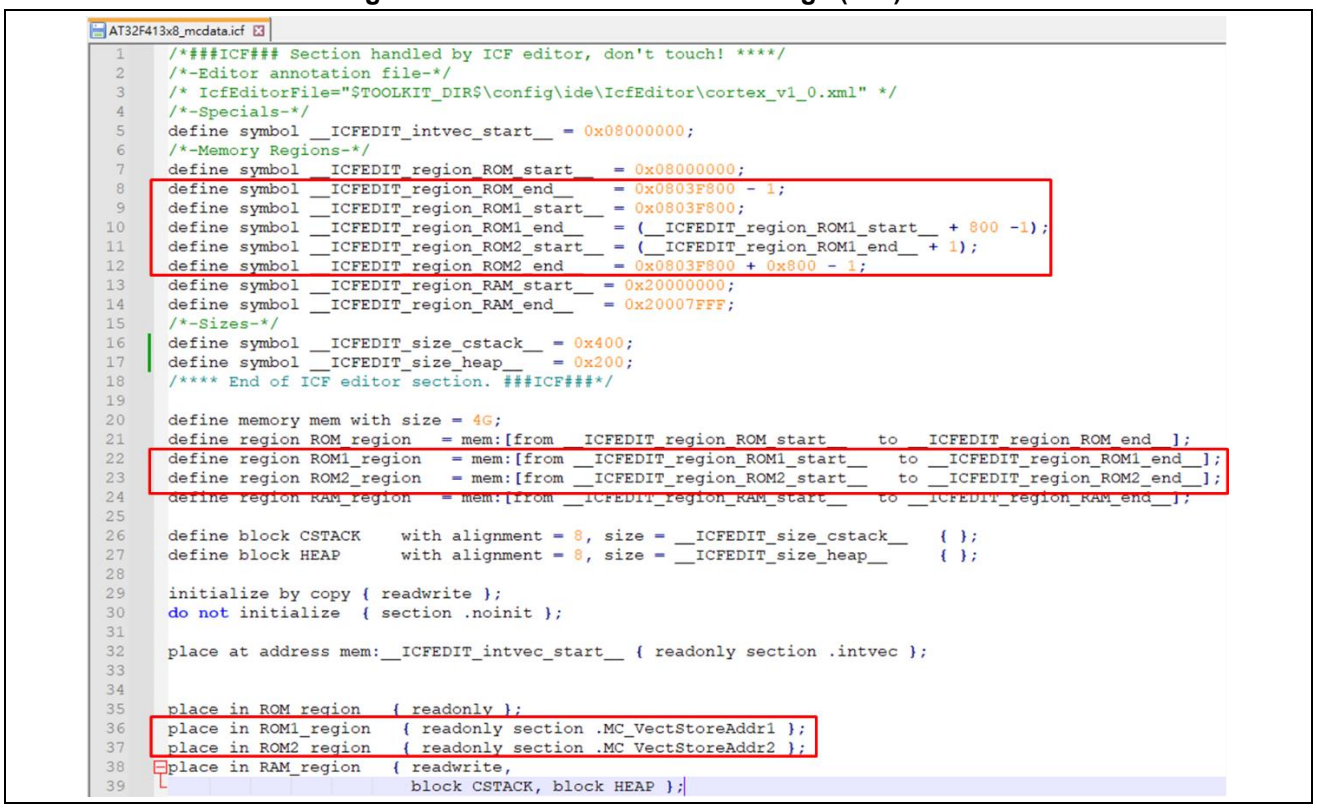


Figure 3. AT32F413RCT7 ROM settings (AT32IDE)

AT32F413xC_FLASH.ld

```
_estack = 0x20008000;    /* end of RAM */

/* Generate a link error if heap and stack don't fit into RAM */
_Min_Heap_Size = 0x200;      /* required amount of heap  */
_Min_Stack_Size = 0x400; /* required amount of stack */

/* Specify the memory areas */
MEMORY
{
FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
MC_DATA (r)     : ORIGIN = 0x0803F800, LENGTH = 0x800
RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
}

/* Define output sections */
SECTIONS
{
  /* The startup code goes first into FLASH */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
  } >FLASH

  .mc_data :
  {
    . = ALIGN(4);
    . = ORIGIN(MC_DATA);
    _MC_VectStoreAddr1 = .;
    *(.MC_VectStoreAddr1);
    . = ALIGN(4);
    . = ORIGIN(MC_DATA) + 800;
    _MC_VectStoreAddr2 = .;
    *(.MC_VectStoreAddr2);
    . = ALIGN(4);
  } >MC_DATA

  /* The program code and other data goes into FLASH */
  .text :
  {
    . = ALIGN(4);
    *(.text)           /* .text sections (code) */
    *(.text*)          /* .text* sections (code) */
    *(.glue_7)         /* glue arm to thumb code */
    *(.glue_7t)        /* glue thumb to arm code */
    *(.eh_frame)
```

Figure 4. AT32F413RCT7 ROM settings (IAR)



3 Quick start guide

Follow the steps below to debug BLDC six-step square wave sensorless control.

Step 1: Set motor control mode and parameters according to the motor configuration (see Section 3.1)

Step 2: Set up the connection with the UI interface (see Section 3.2)

Step 3: Select open loop control to check whether the motor runs properly (see Section 3.3)

Step 4: Current loop control parameter tuning.

(If current loop is not required, enable “WITHOUT_CURRENT_CTRL” in Step 1, omit Steps 4/5/6, and go to Step 7 to adjust the speed voltage loop).

STEP 1 - Use an oscilloscope to determine the current sampling point (see Section 3.4)

STEP 2 - Automatic identification of motor winding parameters (see Section 3.5)

STEP 3 - Current PI controller parameter auto tuning (see Section 3.5)

STEP 4 - IQ tune: fine tuning Kp and Ki values of current loop (see Section 3.6)

Step 5: Sensorless control parameter setting.

1 – Set the initial angle detection startup, constant voltage startup or constant current startup (see Section 3.7)

2 – Set constant current/voltage (& detection with ADC) (see Section 3.8)

Step 6: Speed control parameters tuning (see Section 3.9)

Step 7: Speed voltage control parameters tuning (see Section 3.10)

Step 8: By default, control commands are provided via UI software. To switch to external potentiometer control, adjust the control command source (see Section 3.11).

3.1 Modify motor control mode and parameters

- The *motor_control_drive_param.h* header file allows users to configure the motor drive mode, motor parameters, board parameters and controller parameters.
- Users can configure the desired drive mode according to their hardware and motor configuration, as detailed below.

1. Select to use an internal crystal.

Table 2. Macro definition setting (internal crystal)

Definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)

2. Select a motor development board as needed (for example, AT32-Motor-EV has V1 and V2 versions).

Table 3. Macro definition setting (EVB version)

Definition	Description
AT_MOTOR_EVB_V1	AT-MOTOR-EVB V1.x (include mc_hwio_v1.h file)
AT_MOTOR_EVB_V2	AT-MOTOR-EVB V2.0 (include mc_hwio_v1.h file)

3. Enable complementarity between low-side MOS transistor switch and upper-side PWM as needed. It is recommended to enable this feature in sensorless mode.

Table 4. Macro definition setting (upper-side PWM complementarity)

Definition	Description
COMPLEMENT	Enable complementarity between low-side MOS transistor switch and upper-side PWM (enabled by default; comment to disable)

4. Enable low-side MOS inverted output according to the model of gate driver on hardware. For example, U3315 on AT32-Motor-EV supports inverted output; therefore, this feature is enabled by default; whereas FD6288Q on AT32M412-LV-Motor-EV does not support inverted output; therefore, this feature is disabled by default.

Table 5. Macro definition setting (low-side MOS inverted output)

Definition	Description
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output (enabled by default for AT32-Motor-EV; disabled by default for AT32M412-LV-Motor-EV)

5. Enable EMF voltage compensate according to the motor configuration.

Table 6. Macro definition setting (EMF voltage compensate)

Definition	Description
EMF_COMPENSATE	EMF voltage compensate (disabled by default; define to enable)

6. In this demo project, it is sensorless control mode by default. There is no need to modify this macro definition, and it is reserved for identification by code.

Table 7. Macro definition setting (sensored/sensorless method)

Definition	Description
HALL_SENSORS	Hall sensor
SENSORLESS	Sensorless control mode (general-purpose)

7. Select to enable BEMF detection with ADC or comparator.

Table 8. Macro definition setting (BEMF detection)

Definition	Description
BLDC_SENSORLESS_ADC	BEMF detection with ADC in six-step square wave sensorless control mode
BLDC_SENSORLESS_COMP	BEMF detection with comparator in six-step square wave sensorless control mode

8. Enable current loop control as needed.

Table 9. Macro definition setting (voltage control without current loop)

Definition	Description
WITHOUT_CURRENT_CTRL	Voltage control without current loop (disabled by default; uncomment to enable)

9. Enable low-speed voltage control as needed.

Table 10. Macro definition setting (low-speed voltage control)

Definition	Description
LOW_SPEED_VOLT_CTRL	Low-speed voltage control (enabled by default; comment to disable)

10. Phase change at 30-degree angle delay after zero-crossing detection. Enable phase advance to allow phase change at 0~29-degree angle delay (dependent on the EMF_PHASE_ADV_SPD setting). It is recommended to enable phase advance when switching to high speed mode to avoid phase change error due to excessive long phase change delay time.

Table 11. Macro definition setting (phase advance)

Definition	Description
PHASE_ADVANCE	Phase advance (disabled by default; uncomment to enable)

11. Set this macro definition when INIT_ANGLE_STARTUP is enabled; otherwise, omit this step. When the initial angle detection startup is enabled, the detection method is affected by the large cogging torque. Therefore, users need to set this macro definition according to the motor configuration.

Table 12. Macro definition setting (large cogging torque)

Definition	Description
LARGE_COGGING	Large cogging torque (disabled by default; uncomment to enable)

12. Set this macro definition when ALIGN_AND_GO_STARTUP or OPENLOOP_STARTUP is enabled; otherwise, omit this step. Select constant current or constant voltage startup when switching to angle feedback closed loop control (see Section 3.7).

Table 13. Macro definition setting (constant current /voltage startup)

Definition	Description
CONST_CURRENT_START	Constant current startup (six-step square wave sensorless control)
CONST_VOLTAGE_START	Constant voltage startup (six-step square wave sensorless control)

13. Initial rotor position estimation mode in sensorless control mode (see Section 3.7).

Table 14. Macro definition setting (initial rotor position estimation)

Definition	Description
INIT_ANGLE_STARTUP	Initial angle detection startup in sensorless control mode (general-purpose)
ALIGN_AND_GO_STARTUP	Align and go startup in sensorless control mode (general-purpose)
OPENLOOP_STARTUP	Open loop startup in sensorless control mode (general-purpose)

14. Set this macro definition when BLDC_SENSORLESS_COMP is enabled; otherwise, omit this step. The continuous sampling mode is enabled to detect BEMF zero-crossing for several times in one PWM period, thus to improve detection accuracy. When it is disabled, detect BEMF zero-crossing only once in one PWM period.

Table 15. Macro definition setting (continuous sampling mode)

Definition	Description
EMF_CONTINOUS_SAMPLE	Continuous sampling mode for BEMF zero-crossing detection (disabled by default; uncomment to enable)

15. Set this macro definition when BLDC_SENSORLESS_COMP is enabled; otherwise, omit this step. It is enabled to avoid interference of comparator signals other than the signal of comparator used to detect BEMF (specified phase sequence) based on the phase change angle.

Table 16. Macro definition setting (specific phase sequence)

Definition	Description
SPECIFIC_EACH_EMF_PIN	Avoid comparator signals other than the specific one (enabled by default; comment to disable)

16. Set this macro definition when BLDC_SENSORLESS_ADC is enabled; otherwise, omit this step. It is enabled to improve sampling for BEMF detection with ADC in low speed mode. It is Artery's patent design for pull-up resistance.

Table 17. Macro definition setting (Artery's patent design for pull-up resistance)

Definition	Description
EMF_PULL_UP	Artery's patent design for pull-up resistance, used to improve sampling for BEMF detection with ADC in low speed mode (disabled by default; uncomment to enable)

17. Auto identification of motor winding parameters. It is enabled to automatically identify motor winding RS and LS, and calculate current loop PI controller parameters (see Section 3.5).

Table 18. Macro definition setting (motor winding parameters auto identification)

Definition	Description
MOTOR_PARAM_IDENTIFY	Motor winding parameters auto identification (enabled by default; comment to disable)

18. Enable or disable the Artery motor control UI tool according to user needs.

Table 19. Macro definition setting (Artery motor control UI tool)

Definition	Description
USE_MOTOR_MONITOR	Enable Artery motor control UI tool (enabled by default; comment to disable)

19. Enable or disable the current filtering function based on user needs.

Table 20. Macro definition setting (Current Filtering Function)

Definition	Description
CURRENT_LP_FILTER	Enable the current filtering function (enabled by default; if defined, this function is activated).

- Table 21 presents definitions relating to motor parameters, including number of pole pairs, encoder parameters, Hall sensor parameters, and other relevant parameters.

Table 21. Motor parameter macro definitions

Definition	Description
RS_LL	Motor line-to-line winding resistance (unit: Ω)
LS_LL	Motor line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
ANGLE_INIT_DETECT_DUTY	PWM DUTY value detected at an initial angle (unit: PWM timing base)
KE	Motor KE value (unit: V/rpm)
NOMINAL_CURRENT	Nominal current (unit: ampere)
HALL_LEARN_DIR	Motor rotation direction after HALL self-learning (note [2])
HALL_LEARN_0_STATE	The first state after HALL self-learning (BLDC: output V-shunt high-side PWM and W-shunt low-side ON) (note [2])
HALL_LEARN_1_STATE	The second state after HALL self-learning (BLDC: output V-shunt high-side PWM and U-shunt low-side ON) (note [2])
HALL_LEARN_2_STATE	The third state after HALL self-learning (BLDC: output W-shunt high-side PWM and U-shunt low-side ON) (note [2])
HALL_LEARN_3_STATE	The fourth state after HALL self-learning (BLDC: output W-shunt high-side PWM and V-shunt low-side ON) (note [2])
HALL_LEARN_4_STATE	The fifth state after HALL self-learning (BLDC: output U-shunt high-side PWM and V-shunt low-side ON) (note [2])
HALL_LEARN_5_STATE	The sixth state after HALL self-learning (BLDC: output U-shunt high-side PWM and W-shunt low-side ON) (note [2])

Note [2]: Do not modify in sensorless control mode.

3.2 Connection settings

After the hardware and software are well prepared, set up connection between UI and the control board as follows (see AN0063 for details).

STEP-1
Connect the motor, AT-Link/J-Link and board power supply to the motor development board, and connect USART interface to PC via an USB cable.

STEP-2
Use MDK to compile demo project code, and use J-Link or AT-Link to download to the on-board chip.

STEP-3
Run ArteryMotorMonitor_V2.1.14.exe (software version: V2.1.14); go to File -> Open Project and select ArteryMotorMonitor_V2.1.14.atmcx-> Open.

STEP-4
Click the update icon (1.) of Serial Port and select the corresponding serial port (2.); then click Open(3.) to enable real-time communication, as shown in Figure 5.

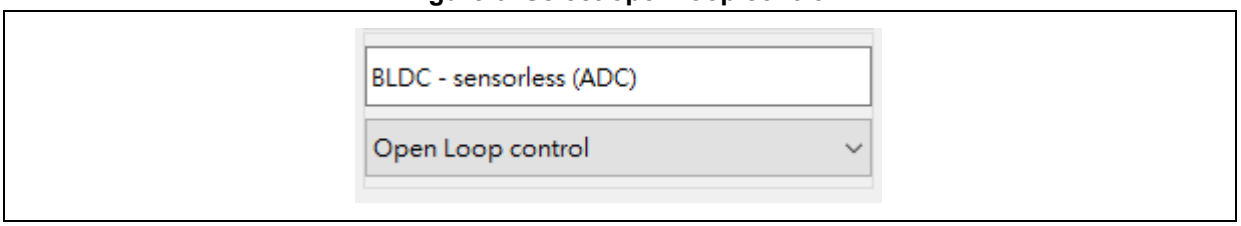
Figure 5. Connection setting

3.3 Six-step square wave open loop control

Select open loop control option to verify that the motor is running properly. This step helps to ensure that the basic control mechanism is functioning before moving to more complex control methods. The open loop voltage and speed are incremented according to the motor running speed and motor phase current value.

STEP-1: Select “Open Loop control”.

Figure 6. Select open loop control



STEP-2: Go to “Tuning Parameters” window (see Figure 7) and set parameters in OpenLoop initial voltage, OpenLoop initial speed, OpenLoop final speed, OpenLoop times and OpenLoop increase volt. Figure 8 shows the open loop control.

Figure 7. Open loop control parameters (BLDC)

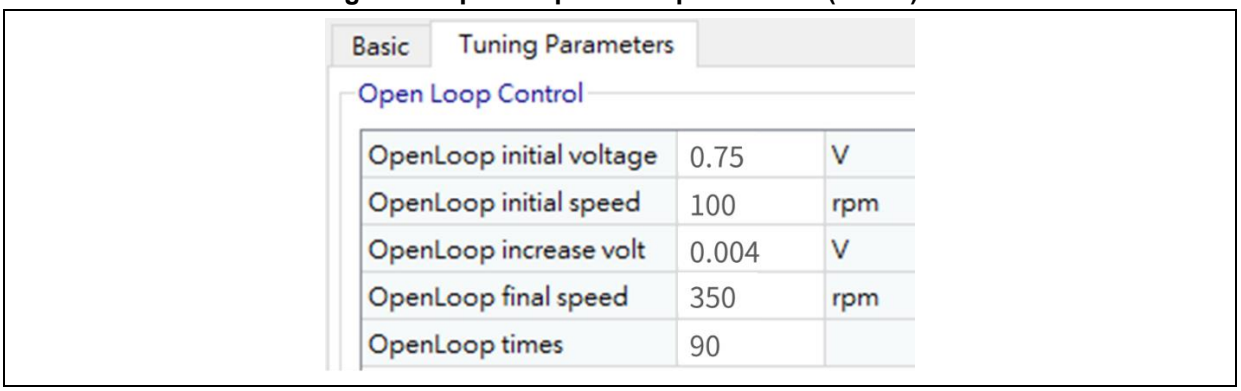
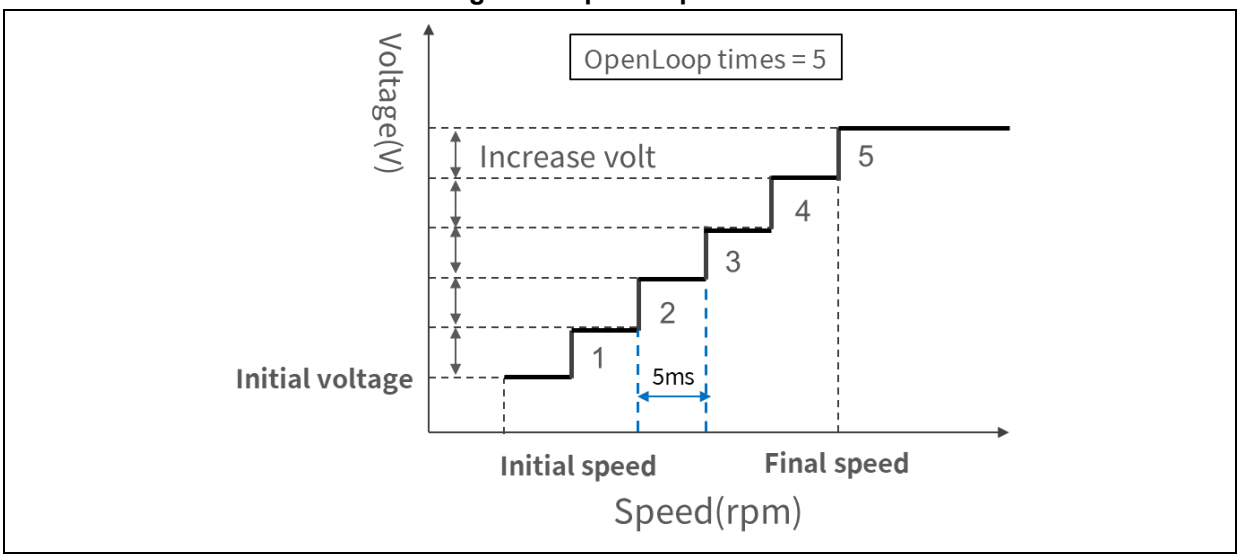


Figure 8. Open loop control



Tips:

1. Initially, set **Openloop increase volt** and **Openloop times** to 0, and set **Openloop final speed** and **Openloop initial speed** to the same value. Then find the appropriate starting voltage and speed.
2. Set the final speed according to the initial speed, and adjust **increase volt** and **Openloop times**. In case of underpower, increase **increase volt** value or **Openloop times** value (it is recommended to be >50).

STEP-3: Click on “Start Motor”, and monitor the current until the motor starts to run properly (set the open loop voltage value appropriately to prevent overheating damage to the motor).

STEP-4: Fill in the parameters into macro definitions in the *motor_control_drive_param.h* file, as shown in Figure 9, and recompile and re-program the code.

Figure 9. Macro definition of open loop control parameters

motor_control_drive_param.h

```
225 /* open loop control */
226 #define OLC_INIT_SPD                (100)      /*!< rpm */
227 #define OLC_FINAL_SPD               (350)      /*!< rpm */
228 #define OLC_TIMES                   (90)       /*!< Accumula
229 #define OLC_INIT_VOLT               (0.75)     /*!< V */
230 #define OLC_VOLT_INC                (0.004)
```

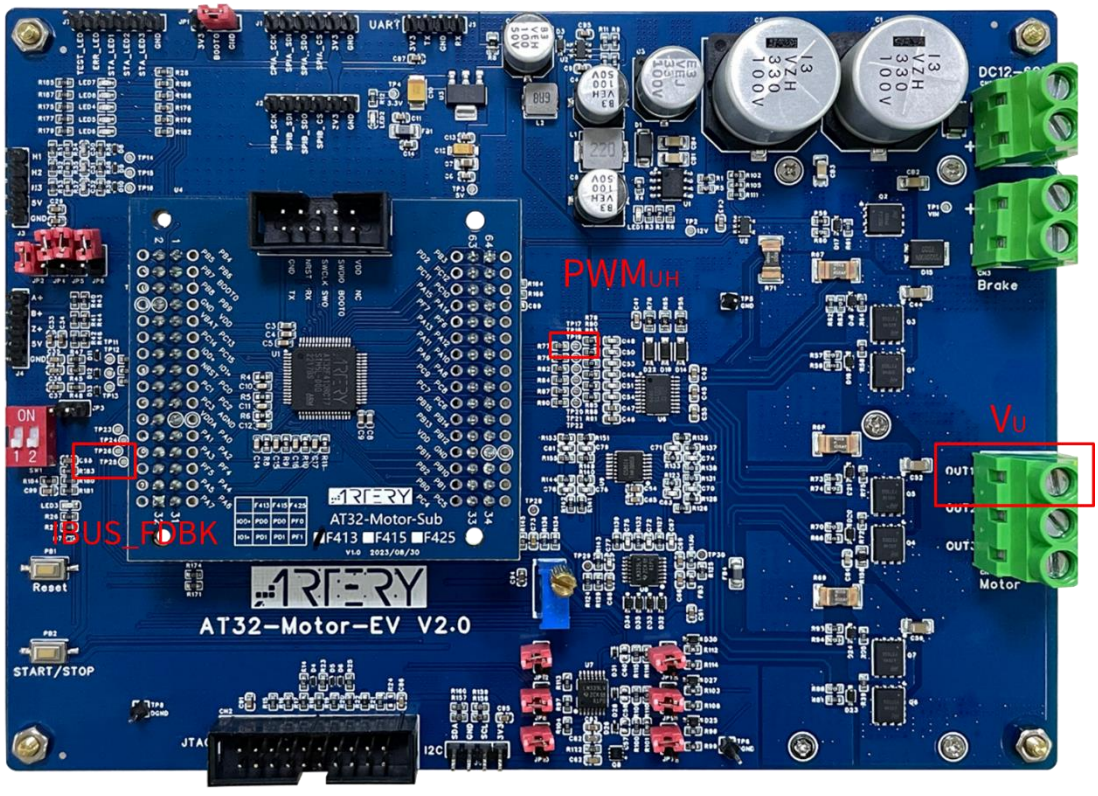
3.4 Current sampling point

Use a 3-channel (at least) oscilloscope to determine the current sampling point. This is crucial for accurate current measurement and control. Signals to be measured are shown in Figure 10.

Oscilloscope channel	Measuring position	Example: AT32-Motor-EVB2.0
CH1	U-shunt output voltage	CN4 OUT1
CH2	Current sensing signal *note [3]	IBUS_FDBK(TP26)
CH3	PWM UH output signal (MCU side)	TIM1_CH1(TP17)

*Note [3]: Because of the need to sense positive and negative currents, the current sense signal has a DC level when the current is 0A, for example, AT32-Motor-EVB2.0 is 0.833V, and CH2 needs to be set to eliminate the zero (-0.83V) when the signal is measured.

Figure 10. Current sampling point signal measurement position (AT32-Motor-EVB2.0)



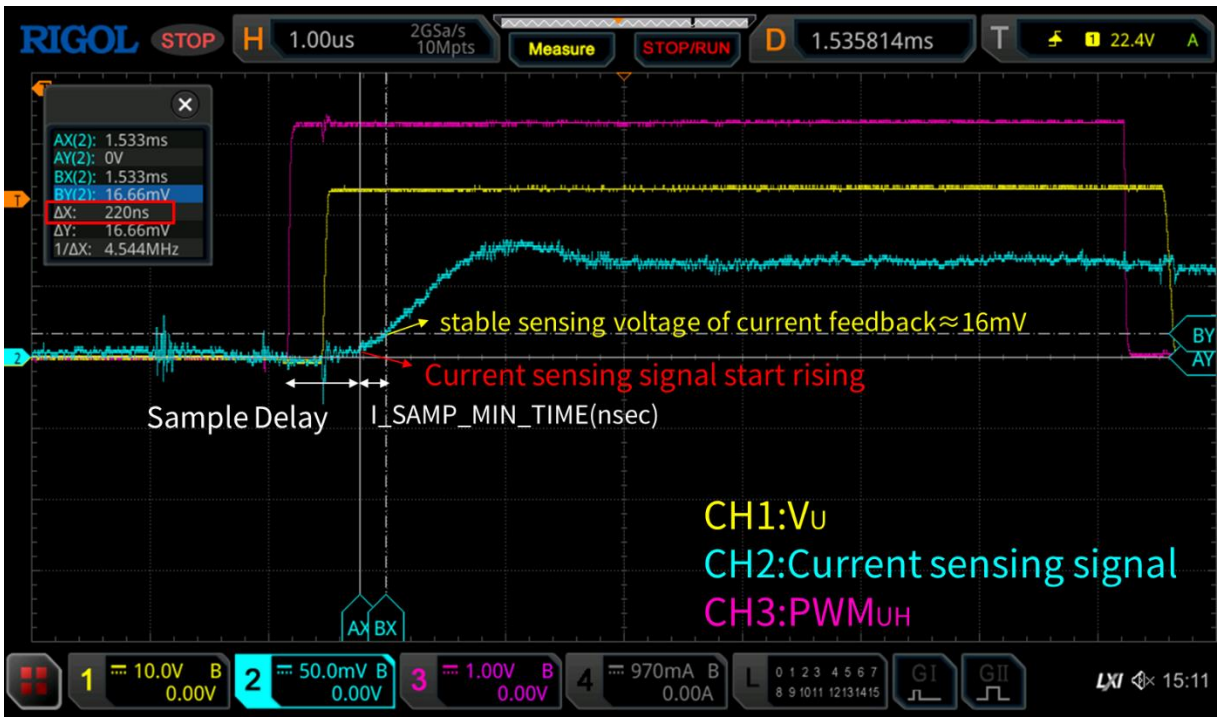
STEP-1: Select six-step square wave open loop control mode to run motor (see Section 3.3).

STEP-2: The rising edge of PWM_UH is used as the trigger point to measure the current sampling delay time, as shown in Figure 6.

Sample Delay = Time difference between PWM UH rise time and the time current sensing signal starts to rise (unit: usec)
I_SAMP_DELAY(nsec) = Sample Delay(nsec) + Dead time(nsec)

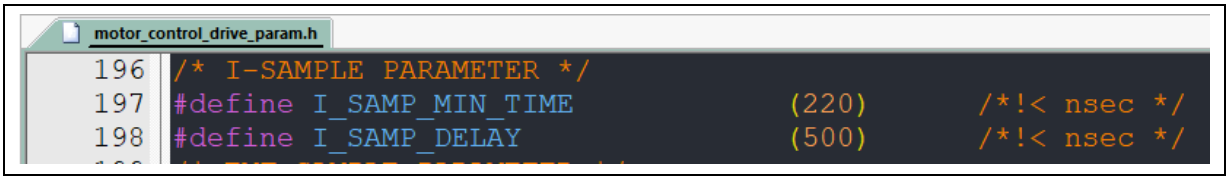
STEP-3: Move the cursor to a stable ADC sampling voltage (about 16mV), and measure the time required for CH2 current sensing signal from 0 mV to 16 mV, that is, I_SAMP_MIN_TIME. As shown in Figure 11, the time is 220 nsec.

Figure 11. Current sampling delay measurement waveform



STEP-4: Fill the delay time value measured in STEP-3 into the macro definition "I_SAMPLE_MIN_TIME" in *motor_control_drive_param.h* file, as shown in Figure 12, and then re-compile and re-program code.

Figure 12. Macro definition of current sampling point

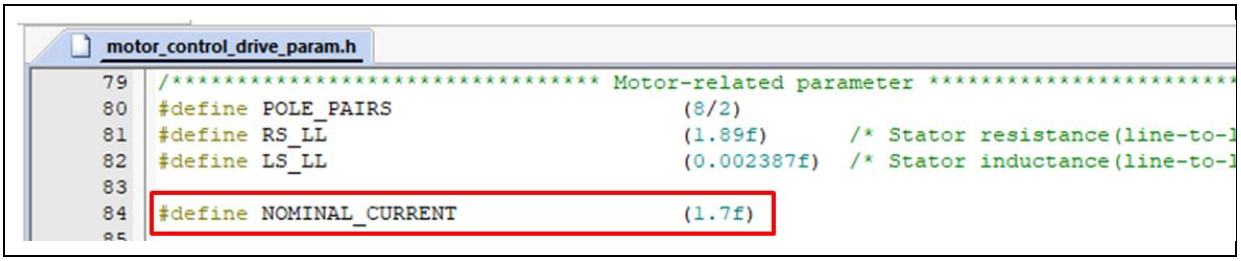


3.5 Auto identification and tuning of parameters

Automatic parameter identification is divided into two parts, namely automatic identification of motor winding parameters and self-tuning of current PI controller parameters.
Auto identification of motor winding parameters are not required if there are RS_LL and LS_LL parameters. Users can directly modify macro definitions of RS_LL and LS_LL in *motor_control_drive_param.h* file.

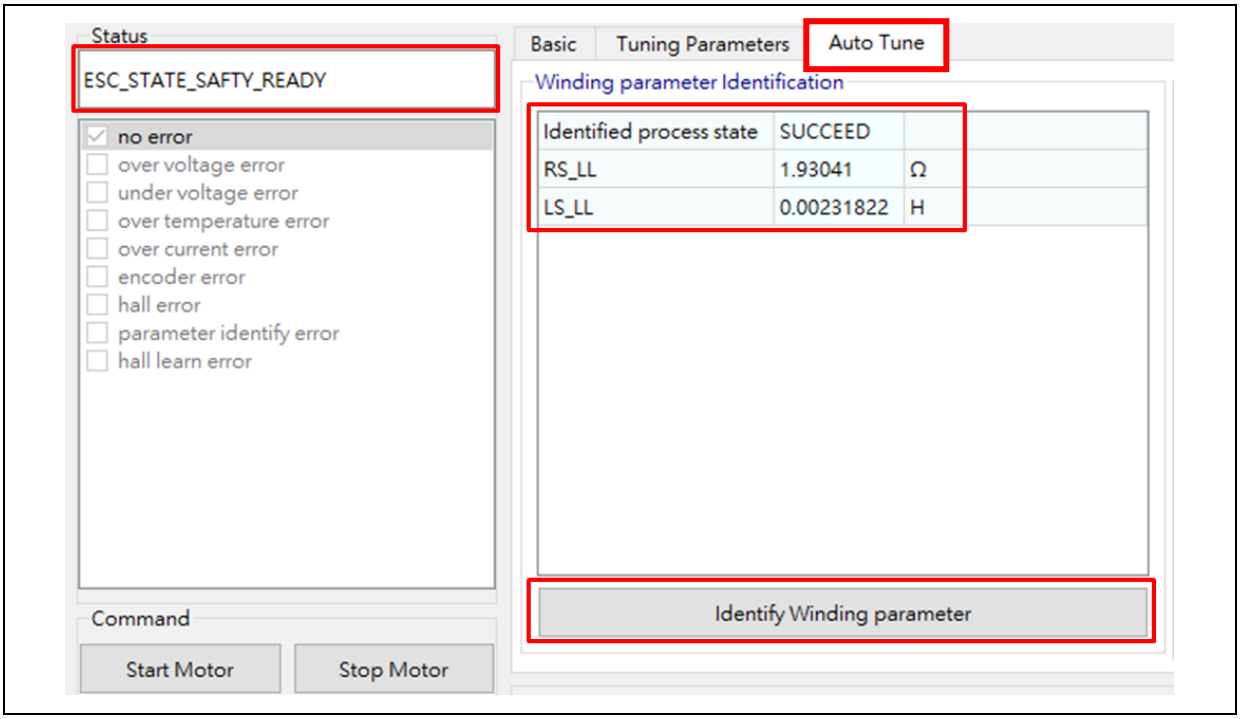
Auto identification of motor winding parameters:
STEP-1: Write the macro definition NOMINAL_CURRENT into *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 13.

Figure 13. Nominal current macro definition



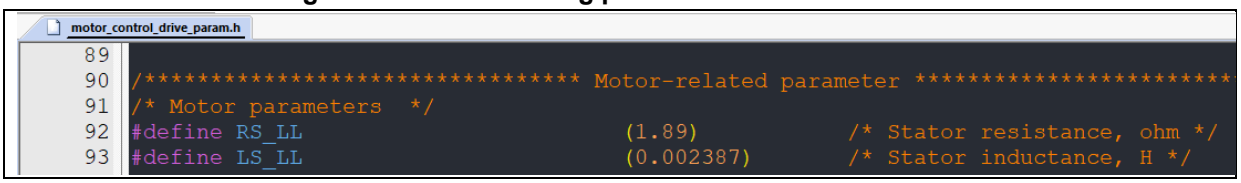
STEP-2: Go to UI --> Auto Tune, and click on “Identify Winding parameter” in “ESC_STATE_SAFETY_READY” status to perform winding parameter identification, as shown in Figure 14.

Figure 14. Identify winding parameters



STEP-3: Check and confirm “SUCCEED” of “Identified process state”; get the RS_LL and LS_LL values and fill them into the corresponding macro definitions in *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 15.

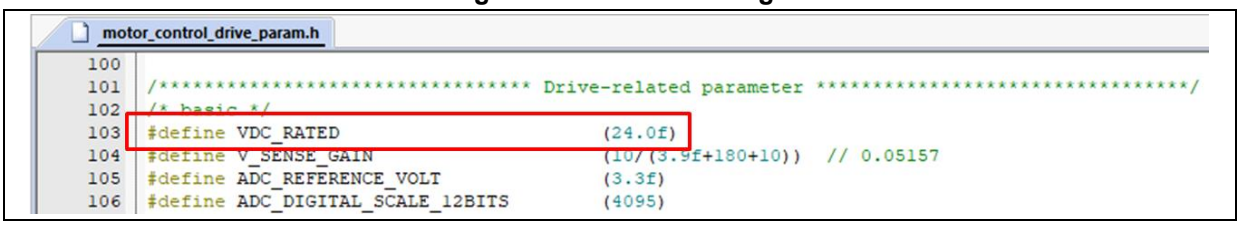
Figure 15. Motor winding parameters macro definition



Current PI controller parameter auto tuning:

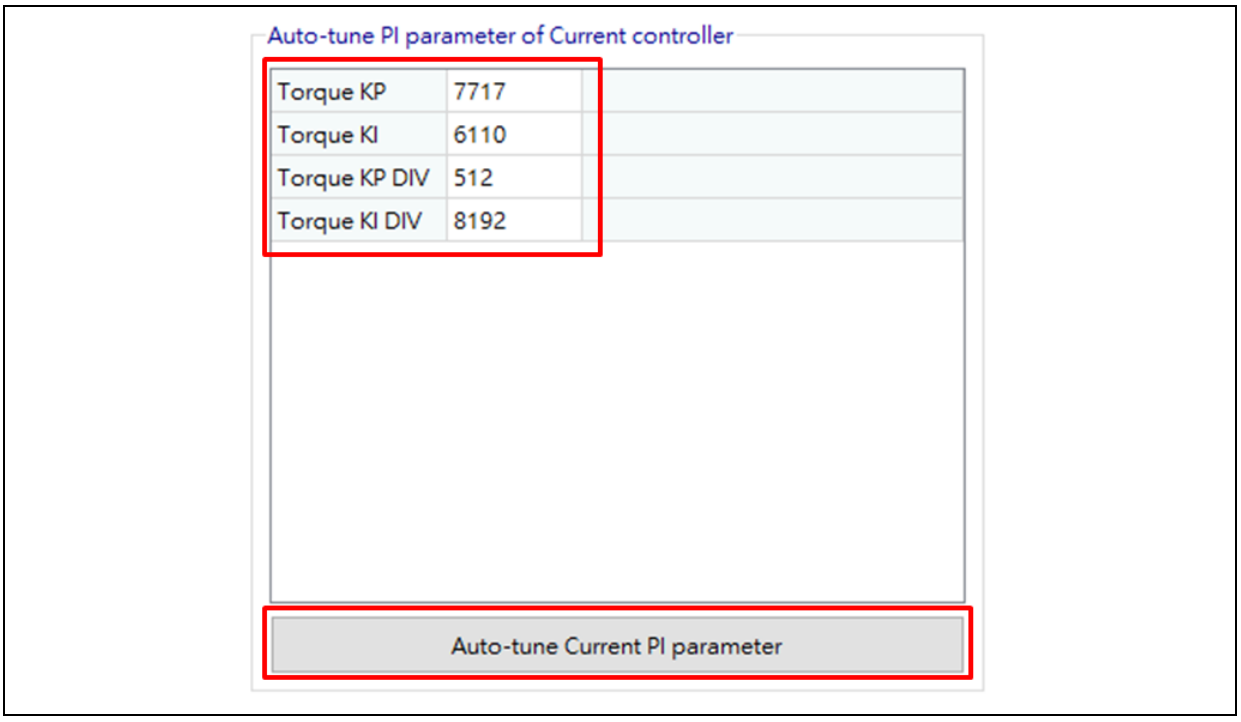
STEP-1: Fill in the macro definition of input DC voltage (VDC_RATED) into the *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 16.

Figure 16. Nominal voltage



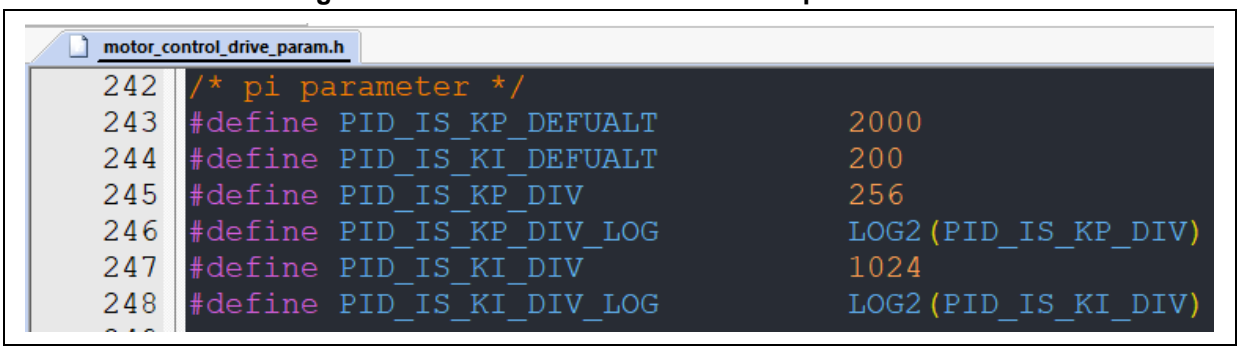
STEP-2: Go to Auto Tune window, and click on “Auto-tune Current PI parameter” in “safety ready” status to perform auto tuning of current PI controller parameters. After auto tuning is completed, current Q axis PI controller parameters are automatically updated, as shown in Figure 17.

Figure 17. Auto tune current PI parameters



STEP-3: Fill the parameters shown in Figure 17 into the corresponding macro definition (motor_control_drive_param.h), then recompile and re-program code, as shown in Figure 18.

Figure 18. Macro definition of current PI parameters



Conversion formula for older versions of motor control library (MC_Library_Project_V2.1.1 and older version):

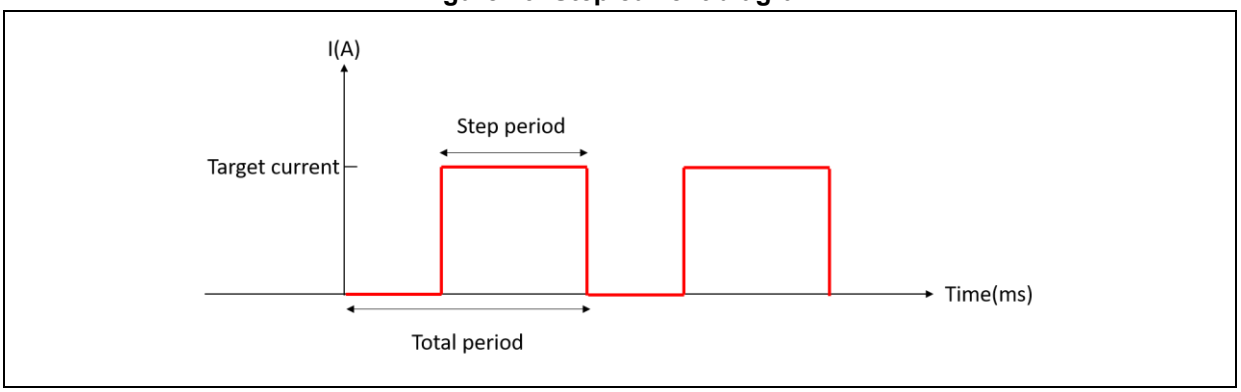
$$PID\_IS\_KP\_DIV\_LOG = \log_2 \text{Torque } KP\ DIV$$

$$PID\_IS\_KI\_DIV\_LOG = \log_2 \text{Torque } KI\ DIV$$

3.6 IQ tune

A step current is generated in IQ Tune mode, as shown in Figure 19. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of Q axis current.

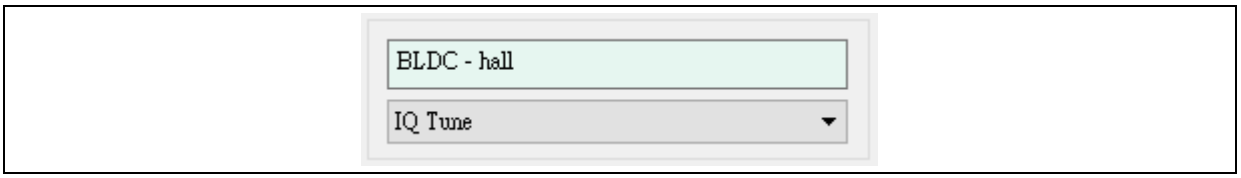
Figure 19. Step current diagram



Follow the steps bellow:

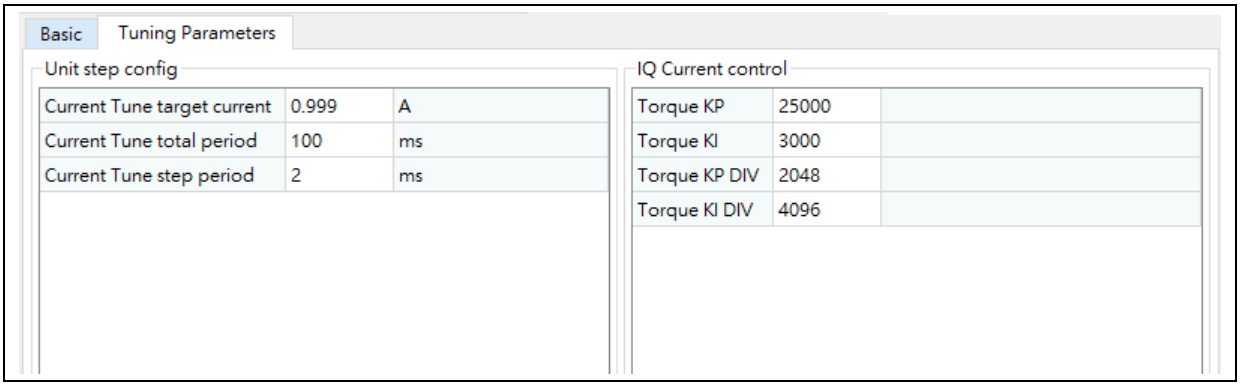
STEP-1: Select “IQ tune”.

Figure 20. Select IQ tune mode



STEP-2: Go to “Tuning Parameters” window to set PID parameters and step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 21.

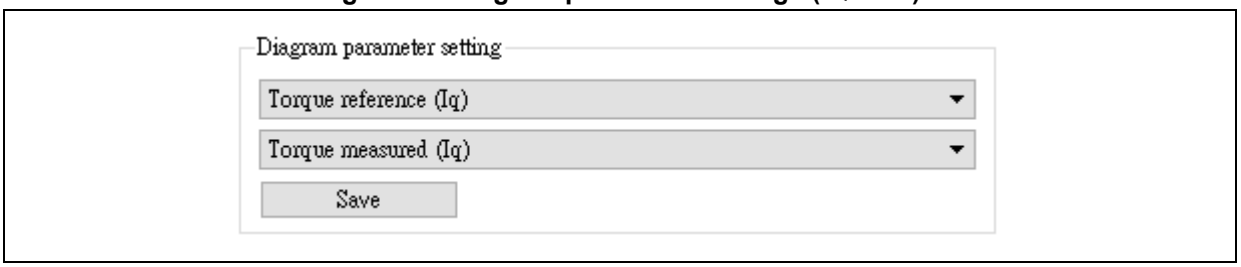
Figure 21. Q-axis current PID and step current parameters



STEP-3: Click on “Start Motor”.

STEP-4: Set “Torque reference(Iq)” and “Torque measured(Iq)” in “Diagram parameter setting”, and then click on “Save”.

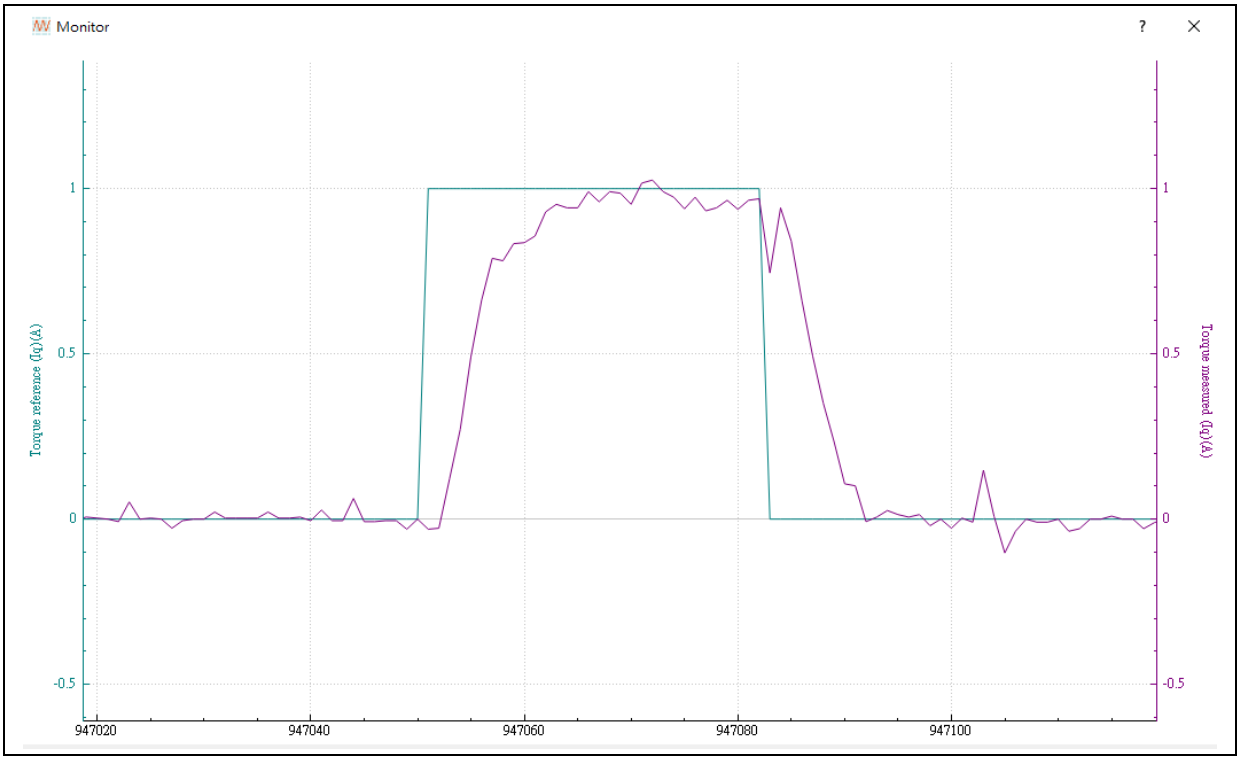
Figure 22. Diagram parameter settings (IQ tune)



STEP-5: Click icon to generate waveforms. For more information, please refer to *AN0063 AT32 Motor Monitor Application Note*.

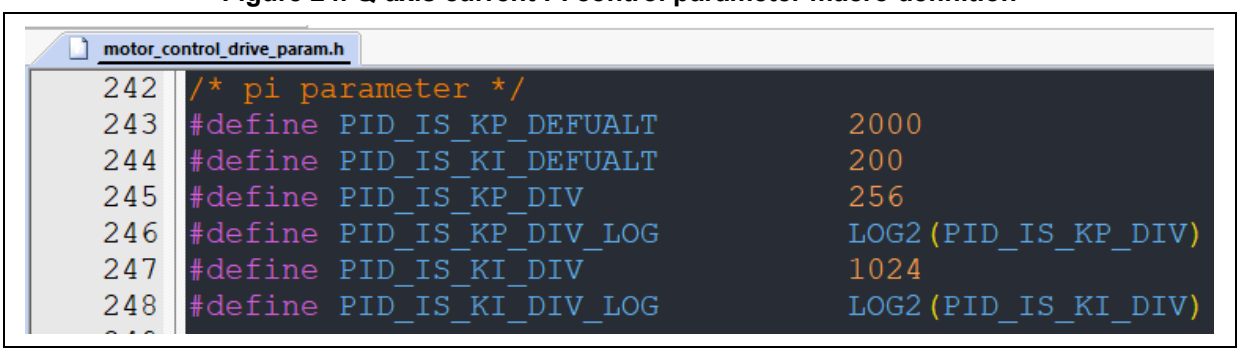
STEP-6: Check whether the current response is as expected, as shown in Figure 23. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 23. IQ tune waveform



STEP-7: After IQ tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 24.

Figure 24. Q-axis current PI control parameter macro definition



3.7 Motor startup method in sensorless mode

The sensorless 120-degree drive method for BLDC is based on determining the rotor position and commutation timing by the zero-crossing points of back EMF. There is no back EMF when motor is in idle state; therefore, other methods are required to get the initial rotor position for motor startup with current vector.

This demo supports the following three startup modes:

1. Initial rotor position startup
   Definition: INIT_ANGLE_STARTUP
   Strength: No reverse
   Weakness: not suitable for motors with low magnetic saturation (high inductance/small rated current) or coreless motor
   Applicable motor type: motors prone to magnetic saturation and equipped with iron-core structures

2. Align and Go startup
   Definition: ALIGN_AND_GO_STARTUP
   Strength: Easy-to-use with few parameters to tune, suitable for most motors
   Weakness: may experience reverse phenomenon
   Applicable motor type: suitable for most types of motors

3. Open loop startup
   Definition: OPENLOOP_STARTUP
   Strength: easy to use
   Weakness: may experience reverse phenomenon, with many parameters to tune
   Applicable motor type: suitable for light-load motors

Users can modify the definition in motor_control_drive_param.h to select the desired startup mode. This demo project selects align and go startup, as shown in Figure 25.

Figure 25. Startup mode definition - six-step square wave sensorless control

```
/* choose how to start up */
#ifdef SENSORLESS
//#define INIT_ANGLE_STARTUP
#define ALIGN_AND_GO_STARTUP
//#define OPENLOOP_STARTUP
#endif
```

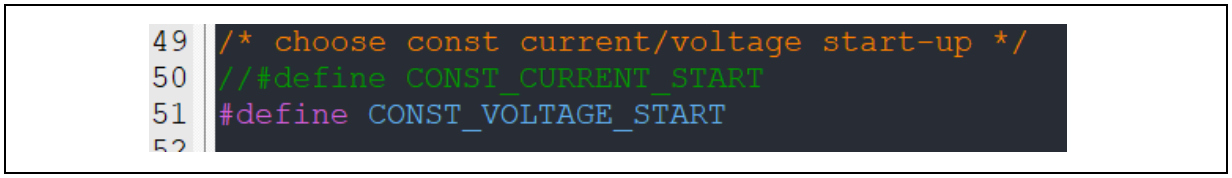
In addition, there are two optional startup control modes, i.e., constant voltage startup and constant current startup.

1. Constant voltage startup
   Definition: CONST_VOLTAGE_START
   Strength: Higher tolerance for commutation errors and higher success rate for starting under light loads
   Weakness: unstable starting torque

2. Constant current startup
   Definition: CONST_CURRENT_START
   Strength: Stable starting torque (suitable for applications requiring torque at startup, e.g., chainsaws)
   Weakness: Prone to generate reverse torque due to misidentification of phase change, resulting in motor vibration

Users can modify the definition in motor_control_drive_param.h to select the desired startup control mode. This demo project selects constant voltage startup, as shown in Figure 26.

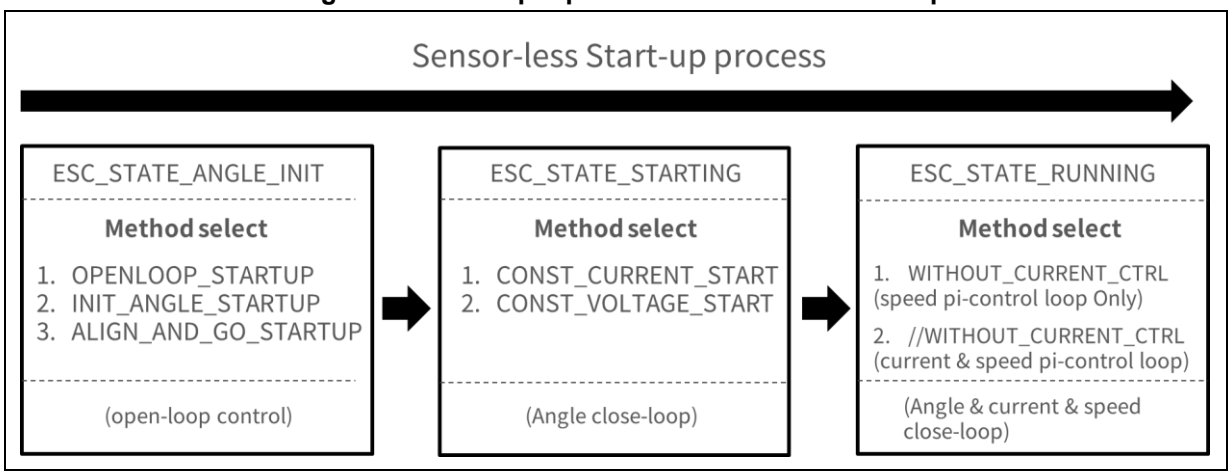
Figure 26. Startup control mode definition – six-step square wave sensorless control



3.8 Sensorless control parameters setting

The sensorless startup process is shown below.

Figure 27. Six-step square wave sensorless startup



Relevant parameters are listed in Table 22.

Table 22. Six-step square wave sensorless startup parameters

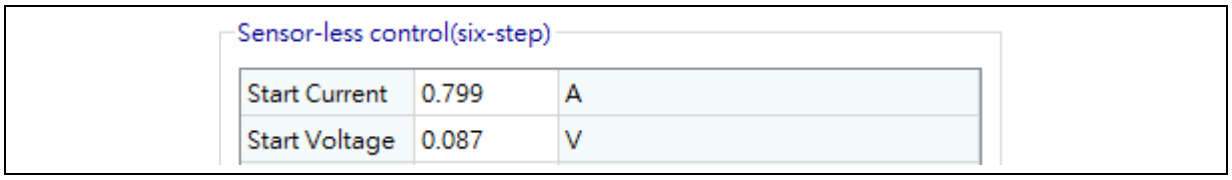
ESC_STATE_ANGLE_INIT		
Mode	**Definition**	**Description**
OPENLOOP_STARTUP	OLC_INIT_SPD	Open loop initial speed (unit: rpm)
	OLC_FINAL_SPD	Open loop final speed (unit: rpm)
	OLC_TIMES	Set the times from initial speed to rise up to the final speed (unit: times)
	OLC_INIT_VOLT	Open loop initial voltage (unit: V)
	OLC_VOLT_INC	Open loop incremental voltage (unit: V)
	OLC_STARTUP_PERIOD	Set the period from open loop startup to closed loop (unit: ms) (for BLDC sensorless use only)
ALIGN_AND_GO_STARTUP	LOCK_VOLT	Rotor alignment voltage before startup (unit: V)
	LOCK_PERIOD	Rotor alignment time before startup (unit: ms)
INIT_ANGLE_STARTUP	ANGLE_INIT_DETECT_DUTY	PWM DUTY value detected at an initial angle (unit: PWM timing base)
	ANGLE_INIT_I_DIFF	Difference of initial angle detection

STARTING		
Mode	**Definition**	**Description**
CONST_CURRENT_START	START_CURRENT	Current value for constant current startup (unit: ampere)
	SENSE_HALL_TIMES	Set phase-change times before open loop enters closed loop (unit: times)
CONST_VOLTAGE_START	START_VOLTAGE	Voltage value for constant voltage startup (unit: V)
	SENSE_HALL_TIMES	Set phase-change times before open loop enters closed loop (unit: times)

RUNNING		
Mode	**Definition**	**Description**
Current loop control	PID_IS_KP_DIV_LOG	Bus current control proportional gain divisor (Q16 mode)
	PID_IS_KI_DIV_LOG	Bus current control integral gain divisor (Q16 mode)
	PID_IS_KP_DEFUALT	Bus current control proportional gain (Q15 mode)
	PID_IS_KI_DEFUALT	Bus current control integral gain (Q15 mode)
Speed loop control (LOW_SPEED_VOLT_CTRL high speed range)	PID_SPD_KP_DIV_DIV	Speed control proportional gain divisor (Q16 mode)
	PID_SPD_KI_DIV_DIV	Speed control integral gain divisor (Q16 mode)
	PID_SPD_KP_DEFUALT	Speed control proportional gain (Q15 mode)
	PID_SPD_KI_DEFUALT	Speed control integral gain (Q15 mode)
Speed voltage loop control (LOW_SPEED_VOLT_CTRL low speed range and WITHOUT_CURRENT_CTRL only)	HYSTERESIS_LOW_SPEED	Minimum speed from current control to switch back to voltage control in low-speed voltage control mode (rpm)
	HYSTERESIS_HIGH_SPEED	Maximum speed from voltage control to switch back to current control in lows-peed voltage control mode (rpm)
	PID_SPD_VOLT_KP_GAIN_DIV	Low-speed voltage control proportional gain divisor (Q16 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
	PID_SPD_VOLT_KI_GAIN_DIV	Low-speed voltage control integral gain divisor (Q16 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
	PID_SPD_VOLT_KP_DEFUALT	Low-speed voltage control proportional gain (Q15 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
	PID_SPD_VOLT_KI_DEFUALT	Low-speed voltage control integral gain (Q15 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)

Different from sensor-based control mode, the sensorless control detects BEMF to estimate the rotor position. Users need to set Start Current or Start Voltage at startup according to the motor configurations, as shown in Figure 28.

Figure 28. Six-step sensorless control parameter setting (BEMF detection with comparator)



1. Start Current/Start Voltage (alternative)
   a) Start current: This is the initial current value for sensorless control during startup, measured in amperes (A).
   b) Start voltage: This is the initial voltage value for sensorless control during startup, measured in volts (V).
   * Set the start current or start voltage, dependent on constant current startup or constant voltage startup.

3.8.1 BEMF detection with ADC

For back EMF detection with ADC, users can set the EMF low speed offset(rising), EMF low speed offset(falling), EMF high speed offset(rising), and EMF high speed offset(falling), as shown in Figure 29 and Figure 30. The correct commutation point is achieved only when the back electromotive forces (back-EMFs) on both sides are symmetrical, as shown in Figure 31.

Figure 29. Six-step sensorless control parameter setting (BEMF detection with ADC)

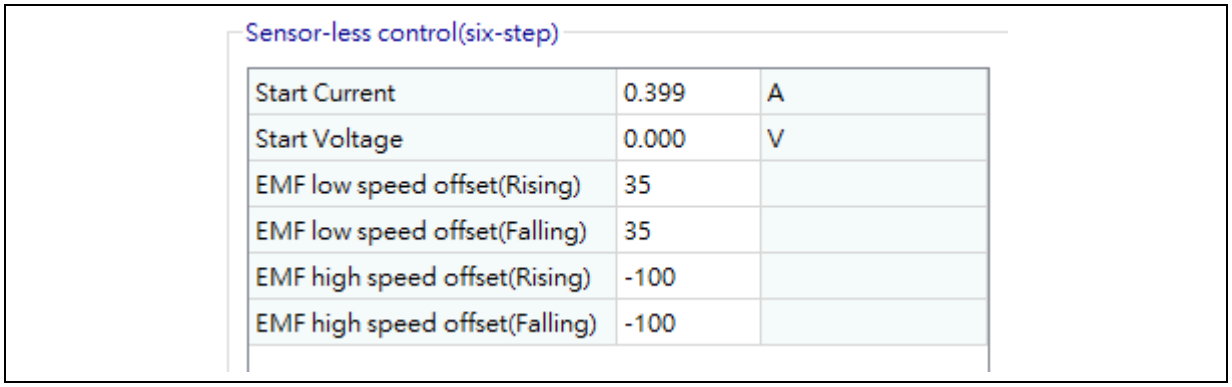
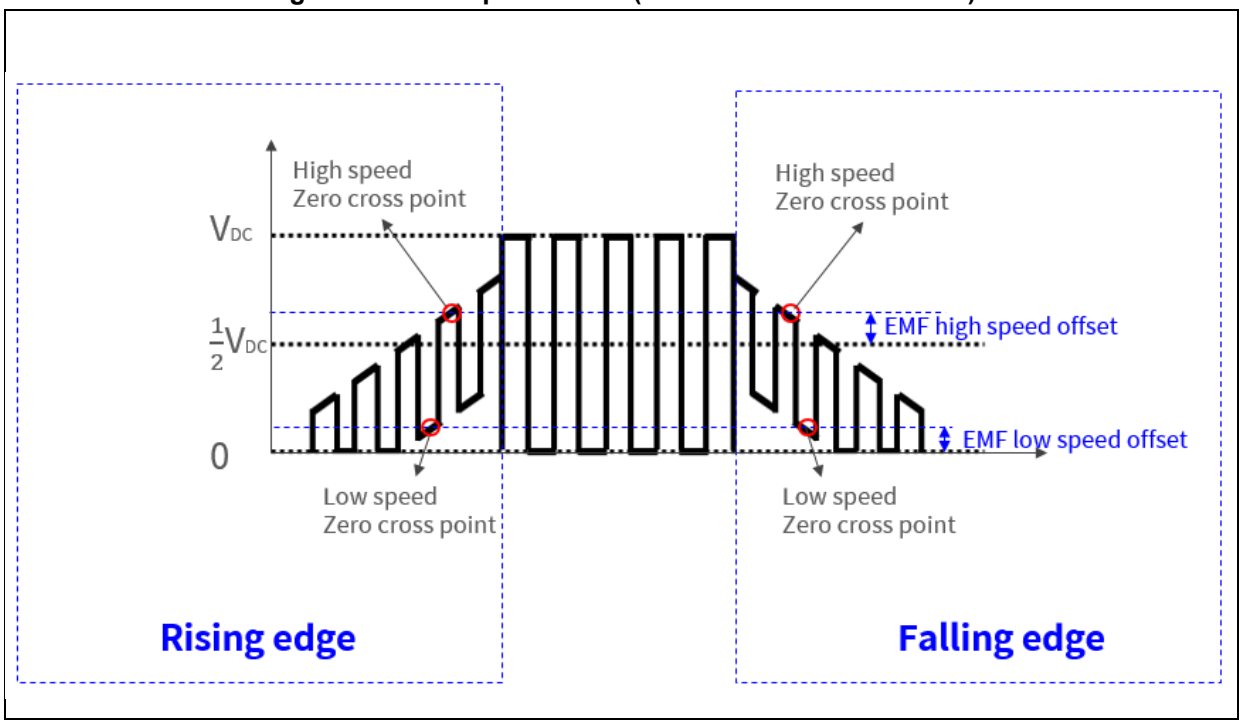


Figure 30. BEMF parameters (BEMF detection with ADC)

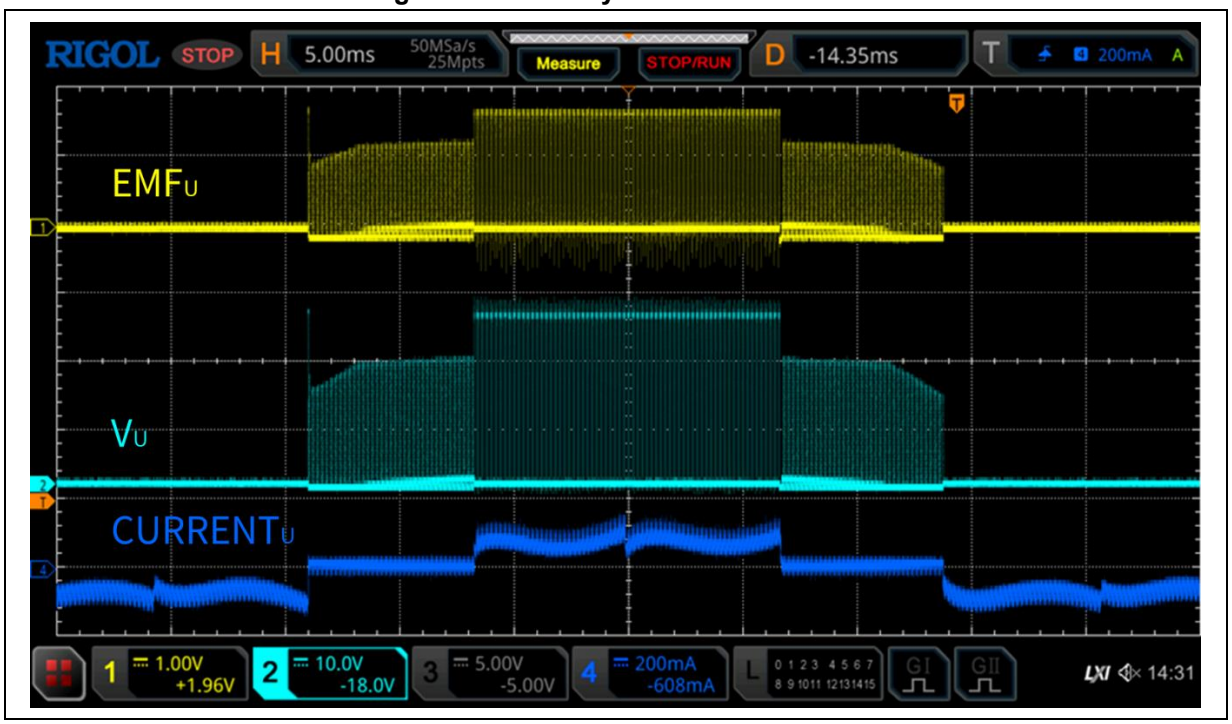


1. EMF low speed offset(rising) and EMF low speed offset(falling) (note [3])
   In low speed mode, detect the BEMF zero-crossing point during PWM OFF period. The “EMF low speed offset” is the level for BEMF zero-crossing point detection (positive edge: rising, negative edge: falling). If the level is set to a positive value (or negative value), the detection threshold moves upwards (or downwards) relative to 0.
   It is the configuration for AT32 MOTOR EVB Board, which needs to be modified according to the board being used. In this example, the rising is 35 and the falling is 35, unit: ADC digital readout of BEMF feedback voltage.

2. EMF high speed offset(rising) and EMF high speed offset(falling) (note [3])
   In high speed mode, detect the BEMF zero-crossing point during PWM ON period. The “EMF high speed offset” is the level for BEMF zero-crossing point detection (positive edge: rising, negative edge: falling). If the level is set to a positive value (or negative value), the detection threshold moves upwards (or downwards) relative to 1/2 VDC.
   It is the configuration for AT32 MOTOR EVB Board, which needs to be modified according to the board being used. In this example, the rising is rising is -100 and the falling is -100, unit: ADC digital readout of BEMF feedback voltage.

   Note [3]: No need to modify for BEMF detection with comparator. These parameters values are related to the hardware circuit of the drive board. There is no need to modify the above configurations if AT32 MOTOR EVB Board is used and the BEMF voltage divider circuit resistance is not changed.

Figure 31. BEMF symmetrical waveform



3.9 Speed loop control

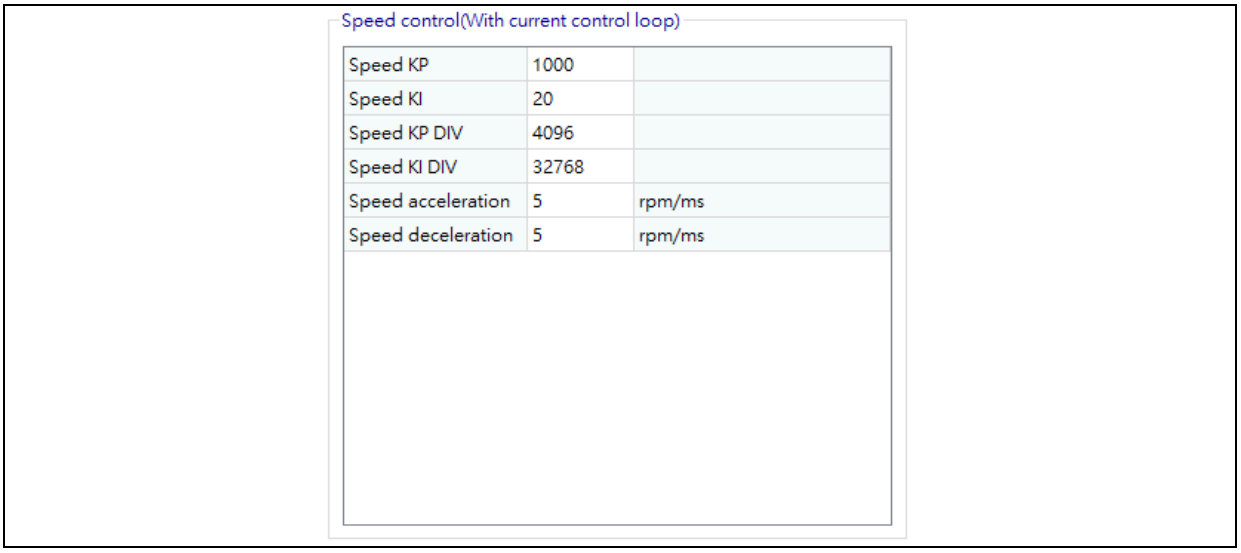
In this mode, users can set the speed PID parameters, acceleration and deceleration, and check the response through waveform.

Follow the below steps:

STEP-1: Select “Speed Control”.

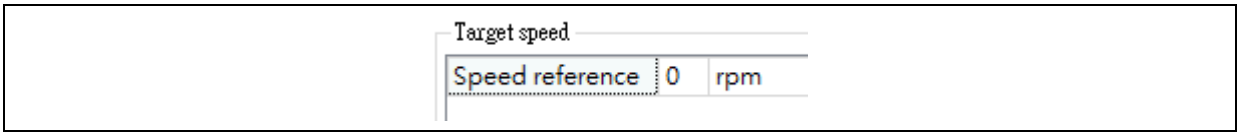
STEP-2: Go to “Tuning Parameters” window, and set speed PID parameters, acceleration and deceleration.

Figure 32. PID parameters, acceleration and deceleration



STEP-3: Select software control as the control source and set the target speed (Speed reference).

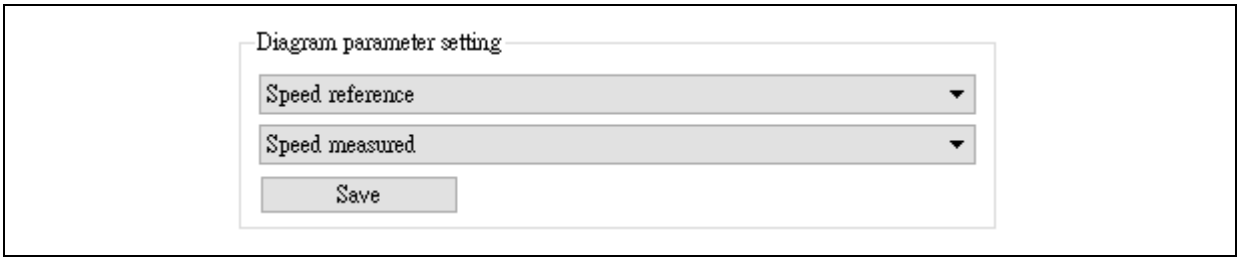
Figure 33. Set target speed



STEP-4: Click on “Start Motor”.

STEP-5: Set “Speed reference” and “Speed measured” in “Diagram parameter setting”, and then click on “Save”.

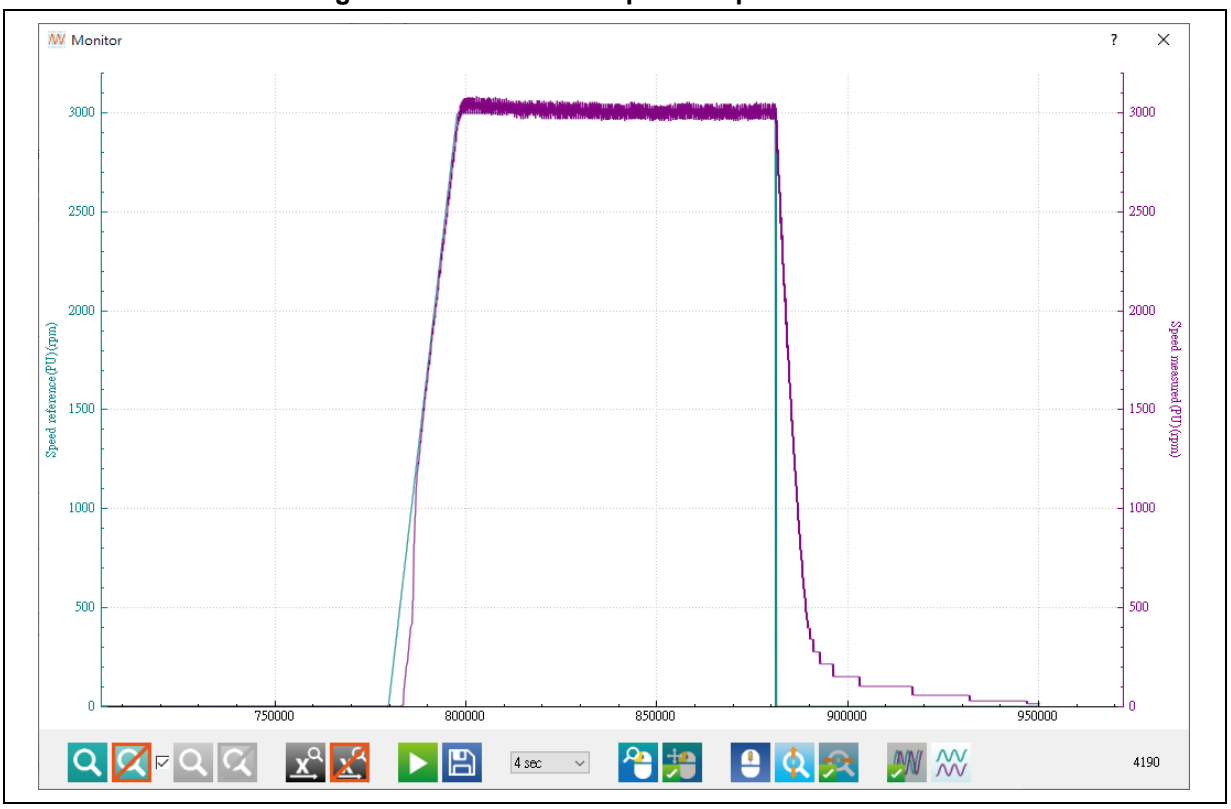
Figure 34. Set channel monitoring parameters (speed control)



STEP-6: Click on the  icon to open the waveform window.

STEP-7: Check whether the speed response is as expected, as shown in Figure 35. If not, click to stop the motor and repeat STEP-2~STEP-7.

Figure 35. Waveform in speed loop control mode



STEP-8: After parameters tuning, fill them into the corresponding macro definitions in *motor_control_drive_param.h* file, and then re-compile and re-program code.

3.10 Speed voltage control

The current is low when the motor runs at a low speed. This demo project provides a low-speed voltage control method to support voltage control at low speed running and switches to closed-loop control when the current becomes high.
The speed voltage control method is used to control output voltage according to the speed difference. It is required when the WITHOUT_CURRENT_CTRL (voltage control only) is enabled or the low-speed range of LOW_SPEED_VOLT_CTRL (low-speed voltage control) is enabled.
In this mode, users can set the speed voltage PID parameters, and check the response through waveform.

STEP-1: Define LOW_SPEED_VOLT_CTRL in *motor_control_drive_param.h* file, as shown in Figure 29.

Figure 36. Definition of low-speed voltage control

motor_control_drive_param.h

```
58 #ifdef HALL_SENSORS
59 /* choose low speed control or not */
60 #define LOW_SPEED_VOLT_CTRL
```

STEP-2: Set the minimum speed (HYSTERESIS_LOW_SPEED) shifting from current control to lows-peed voltage control, and the maximum speed (HYSTERESIS_HIGH_SPEED) shifting from lows-peed voltage control to current control, as shown in Figure 37.

Figure 37. Low-speed voltage control parameter definition

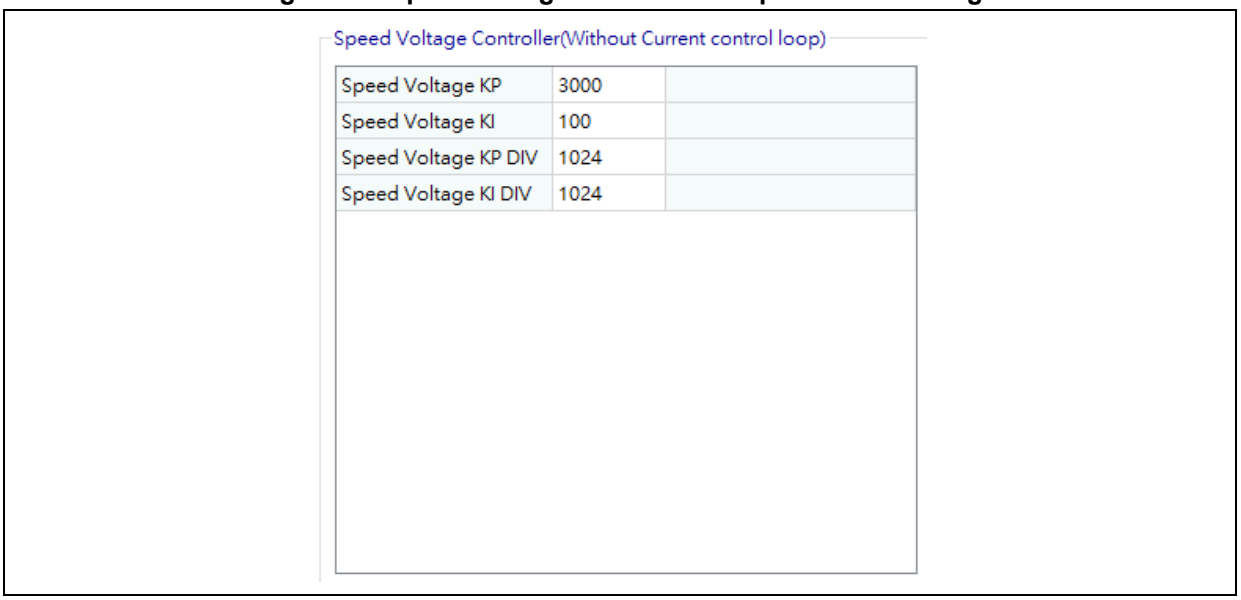
motor_control_drive_param.h

```
253
254 /* low speed voltage control parameter */
255 #define HYSTERESIS_LOW_SPEED       (800)      /*!< rpm */
256 #define HYSTERESIS_HIGH_SPEED      (1000)     /*!< rpm */
257 /* low speed pid parameter for voltage control */
258 #define PID_SPD_VOLT_KP_DEFUALT          3000
259 #define PID_SPD_VOLT_KP_GAIN_DIV_LOG     10
260 #define PID_SPD_VOLT_KI_DEFUALT          100
261 #define PID_SPD_VOLT_KI_GAIN_DIV_LOG     10
```

STEP-3: Select “Speed Control”.

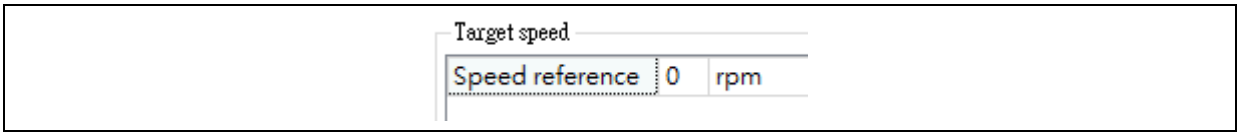
STEP-4: Go to “Tuning Parameters” window and set the speed voltage PID parameters.

Figure 38. Speed voltage controller PID parameter setting



STEP-5: Select “software control” as the control source and set the target speed (Speed reference), for example, 300 rpm.

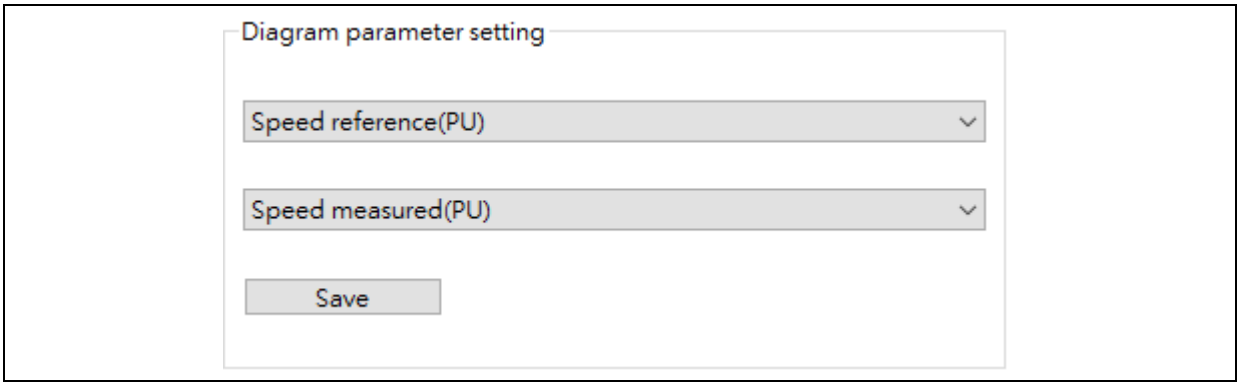
Figure 39. Set target speed



STEP-6: Click on “Start Motor”.

STEP-7: Set “Speed reference(PU)” and “Speed measured(PU)” in “Diagram parameter setting”, and then click on “Save”.

Figure 40. Set channel monitoring parameters (speed control)



STEP-8: Click on the  icon to open the waveform window.

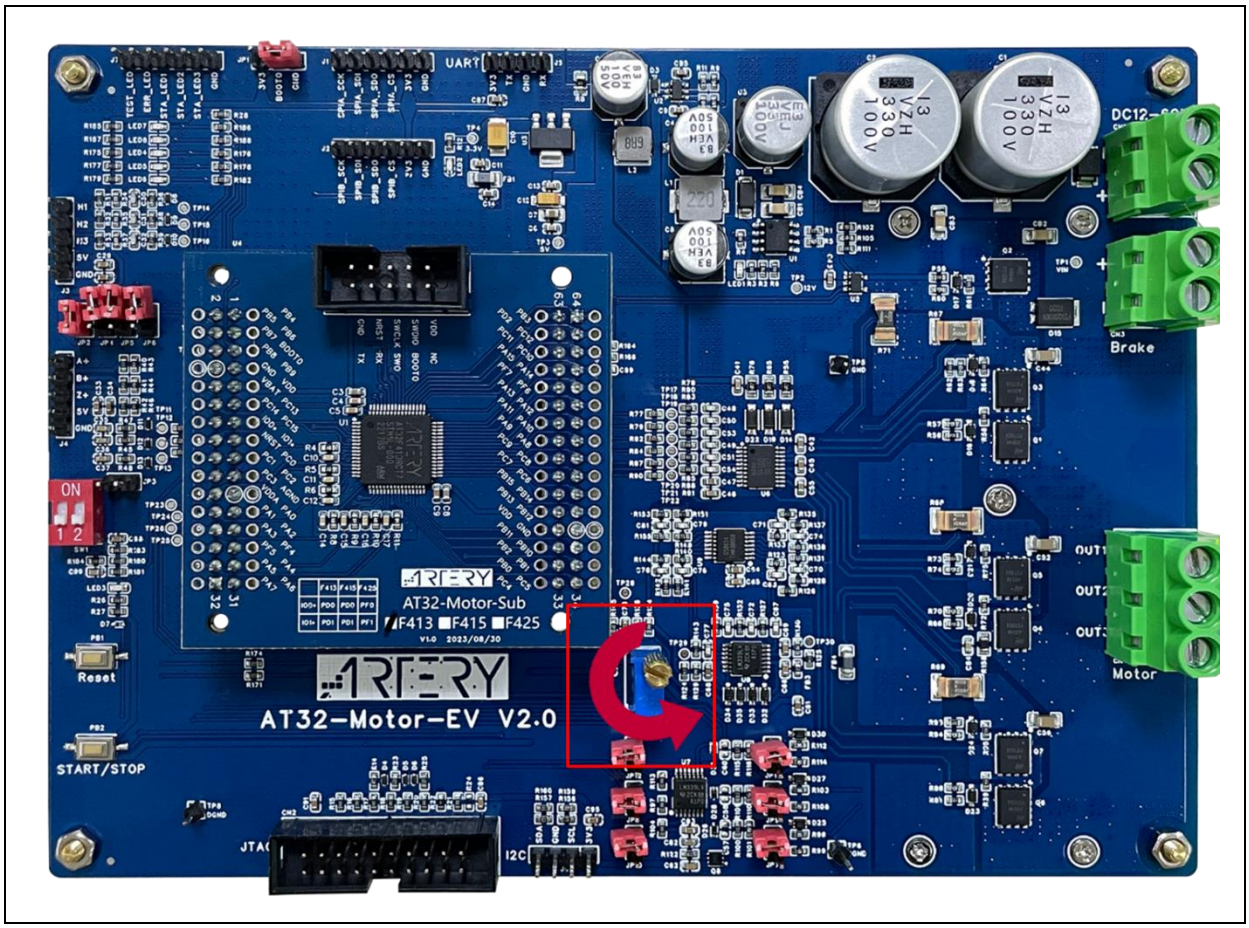
STEP-9: Check whether the speed response is as expected, as shown in Figure 41. If not, click to stop the motor and repeat STEP-4~STEP-9.

Figure 41. Waveform in speed voltage control mode

3.11 External voltage control/software command control

This demo project support two control sources, i.e., external voltage control and software control. The external voltage control mode adjusts speed or torque through external voltage. Users can adjust the potentiometer (as shown in the red box below) to control the command value. The voltage increases when the potentiometer is turned **counterclockwise**. The software control mode inputs speed or torque command value for control purpose. In this demo project, software control mode is set by default.

Figure 42. External voltage control – potentiometer



Users can select the control source by modifying the corresponding definition (CTRL_SOURCE in *motor_control_drive_param.h* file, as shown in Figure 43 or through PC software (see Figure 44, Figure 45 and Figure 46) to set control parameters for different control modes.

Figure 43. Control source definition (software control (upper)/external voltage control (lower))

motor_control_drive_param.h

```
183
184 /********************************* Control-related parameter ************************
185 #define PWM_FREQ                  (30000)                    /*!< Hz */
186 #define MOTOR_CONTROL_MODE        (motor_control_mode)(SPEED_CTRL)
187 #define CTRL_SOURCE               (ctrl_source_type)(CTRL_SOURCE_SOFTWARE)
188 #define UI_UART_BAUDRATE          (1500000UL)
189 #define UI_SAMPLE_CYCLE           (1)
```

```
motor_control_drive_param.h
183
184 /********************************* Control-related parameter ***********************
185 #define PWM_FREQ                    (30000)                          /*!< Hz */
186 #define MOTOR_CONTROL_MODE          (motor_control_mode)(SPEED_CTRL)
187 #define CTRL_SOURCE                 (ctrl_source_type)(CTRL_SOURCE_EXTERNAL)
188 #define UI_UART_BAUDRATE            (15000000UL)
189 #define UI_SAMPLE_CYCLE             (1)
```

1) External voltage control

Go to “Control source” and select “external voltage control” to adjust the speed or torque through external voltage. The target speed or torque reference field displays the converted speed or torque at the current control voltage.

Note: This field cannot be modified in the external control source mode.

Figure 44. Speed control mode (external voltage control)

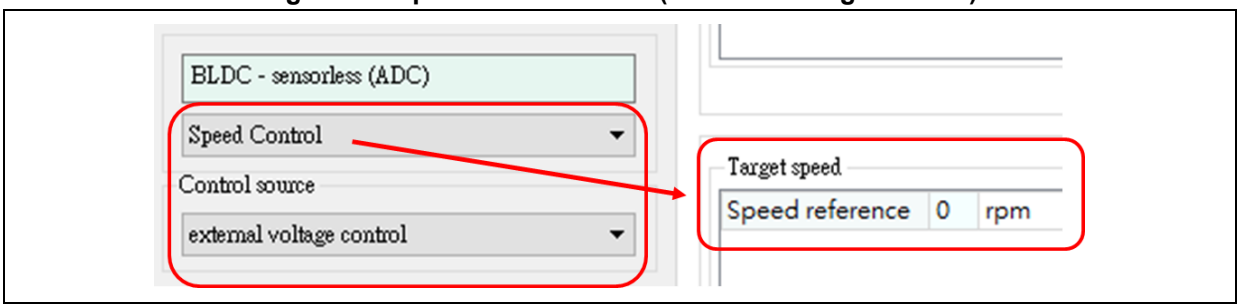
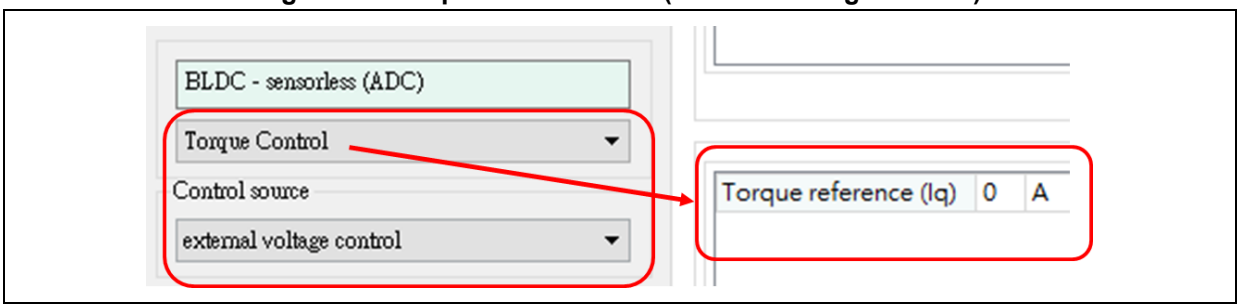


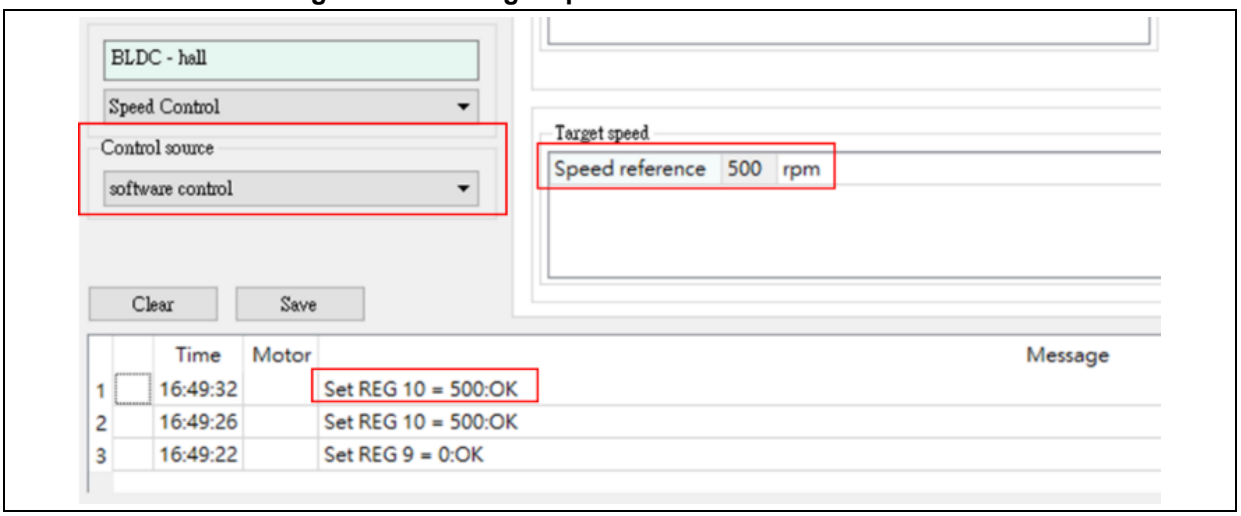
Figure 45. Torque control mode (external voltage control)



2) Software control

By default, the software control mode is enabled. You can switch to software control mode by selecting "Software control" from the source selection dropdown menu, as shown in Figure 46. In this mode, the speed/torque of the motor is adjusted by modifying the target speed/torque via the UI interface. Double-clicking on this field allows you to change the value. Once the setting is complete, a message indicating the successful setting will be displayed in the field below.

Figure 46. Set target speed in software control mode



4 Revision history

Table 23. Document revision history

Date	Version	Revision note
2024.05.17	2.1.0	Initial release.
2024.12.27	2.1.1	Added Section 3.10 “Speed voltage control.
2025.10.09	2.1.3	Modified Flash ROM configuration in Table 1, and added Section 3.8.1 about the BEMF detection with ADC.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.